

Proyecto 1. Valor 1%

Temas de Estudio

- Tema 1. Introducción a los compiladores
- Tema 2. Un traductor simple orientado a la sintaxis

Objetivo

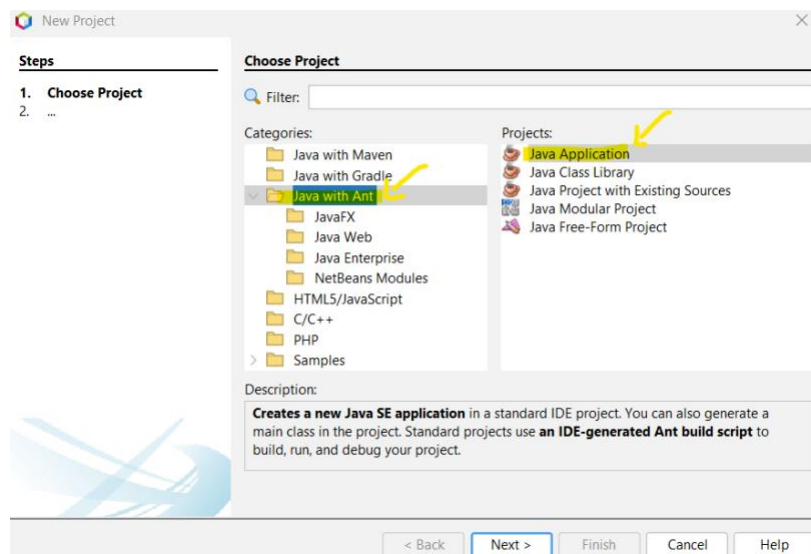
Aplicar los conocimientos adquiridos en el curso, centrándonos en la programación relacionada con los conceptos de compiladores, intérpretes, para validar a nivel de análisis léxico y la sintaxis, de un código fuente el cual es formado por entrada de cadenas de textos.

Software de Desarrollo

El proyecto debe desarrollarse utilizando únicamente el lenguaje de **programación Java** y el entorno de desarrollo **integrado (IDE) Apache NetBeans**, deberá instalar la versión más reciente o tres versiones antes de la última, no se aceptan entregas con proyectos desarrollados en versión obsoleta de Netbeans 8.2, en este enlace oficial podrá encontrar el instalador <https://dlcdn.apache.org/netbeans/netbeans-installers/>. Cuando ya lo tenga instalado debe considerar que hay tres maneras de crear proyectos en Apache Netbeans:

- Ant
- Gradle
- Maven

Solo se debe utilizar la primera manera que se refiere a proyectos tipo Ant, debido a que son proyectos que se moldean a nuestro aprendizaje y además tiene solución más liviana, en la siguiente imagen se muestra cómo crear un proyecto:



y deberá crear específicamente un proyecto de tipo "**Java con Ant**" en modo carácter. No está permitido el uso de interfaces gráficas. Además, no se permiten proyectos del tipo "Java con Maven" ni "Java con Gradle". Sino se cumple con lo indicado, el proyecto no será aceptado.

Desarrollo

PASCAL fue presentado en el año 1970, por Niklaus Wirth, PASCAL fue diseñado para forzar la implementación de programas bien estructurados, claros y reducir el número de errores durante el desarrollo. Estas características lo convirtieron pronto en la referencia para la docencia de programación en la Universidad.

PASCAL es un lenguaje fuertemente tipado, con código dividido en fragmentos con semántica propia (funciones o procedimientos) y con variables que han de ser declaradas, como paso previo a su uso, impidiéndose la interpretación o conversión de valores como por ejemplo hace C.

- Para este proyecto se requiere realizar un análisis que identifique los tokens o componentes léxicos en el código PASCAL, con el fin de verificar su uso adecuado y las estructuras sintácticas, de acuerdo con las reglas especificadas en la rúbrica del proyecto.
- El intérprete que usted desarrollará en Apache NetBeans, utilizando el lenguaje Java en un proyecto tipo "Java con Ant", debe ser completamente independiente de una ruta física. Esto significa que el programa podrá ejecutarse en cualquier máquina y directorio sin depender de configuraciones específicas. Para su ejecución, se utilizará la consola de Windows (CMD), ejecutando el archivo compilado (.jar).
- El proyecto implicará la lectura de un archivo con extensión .pas que contenga un ejemplo de código en PASCAL.
- Para que este archivo .pas pueda ser leído, no deberá depender de un nombre específico ni de una ruta estática. En su lugar, deberá estar ubicado en el mismo directorio que el archivo compilado .jar.
- Es necesario detectar y registrar los posibles errores en un archivo de salida con extensión .err, conforme a las especificaciones detalladas en la rúbrica.
- No se permite el uso de librerías de terceros, entendidas como fragmentos de código desarrollados por otras personas y encapsulados en componentes que podrían servir como referencia para la solución del problema.
- Se le recomienda que por favor se tome su tiempo para revisar y comprender las especificaciones señaladas en la rubrica de evaluación.

Código fuente:

- El código que se muestra a continuación corresponde un calendario y está desarrollado en el lenguaje programación PASCAL.
- El código no tiene ningún error y lo puede ejecutar online en el siguiente sitio que interpreta código PASCAL https://www.onlinegdb.com/online_pascal_compiler
- Para cambiar de mes presione las teclas ← (flecha izquierda) y → (flecha derecha).
- Para cambiar de año presione las teclas ↑ (flecha arriba) y ↓ (flecha abajo).
- Para terminar el programa presione la tecla ESC.
- Usted puede trabajar a partir de dicho código para realizar lo que se le solicita, **pero tenga en cuenta que el código, será modificado por el tutor (a)** para agregar errores según se mencionan en la rúbrica para poder comprobar si el estudiante realizó la validación que se le solicitó, el cambio abarca desde el código fuente del archivo .pas, hasta el cambio en el nombre del archivo a revisar.
- A continuación, se especifica el código con el que se estará trabajando en este proyecto, por favor tome en consideración que dicho código está disponible en un archivo descargable en la plataforma “Moodle” con la extensión **.pas**

Página 4 de 15

Evaluación:

#	Requerimientos para evaluar
1	Lectura archivo: se debe leer un archivo de código PASCAL con extensión .pas no debe leer otra extensión que no sea la indicada, <u>de lo contrario no está cumpliendo con este punto.</u> • La ejecución del ejecutable .jar debe poder realizarse desde la consola CMD o símbolo del sistema. • En la plataforma se provee un ejemplo en dicho formato de extensión indicado, debe utilizar ese. • Al leer el archivo su programa debe aceptar espacios en blanco, líneas en blanco, cualquier espacio en general que se encuentre y que se distinga entre mayúsculas y minúsculas. Es decir, el formato original del archivo debe ser respetado en su totalidad. • La lectura del archivo debe ser cargada en la memoria del programa para realizar los diferentes análisis y verificaciones propuestos en este proyecto. • Es importante, que el archivo .pas, no dependa de un nombre en específico para poder ser leído, y que no dependa de una ruta estática o física ni configuraciones personalizadas por el desarrollador (estudiante), <u>de lo contrario no está cumpliendo con este punto.</u> Con este punto se refiere a que por medio del paso de parámetros por la consola de Windows o CMD, el estudiante recibe un nombre determinado y ese es el archivo que procesará en el proyecto. Se recuerda que en donde se encuentre el compilado de su programa .jar en la misma carpeta esté el archivo .pas a leer

2	Manejo de errores: A partir del archivo que se leyó se debe crear otro archivo con extensión .err - El nombre del nuevo archivo debe conservar el nombre original y agregarle un guion con la descripción “-errores.err” el archivo debe verse de esta manera: “nombreArchivo-errores.err” donde “nombreArchivo” depende del nombre que tenga dicho archivo de entrada .pas - Se debe respetar el contenido y formato del archivo original .pas, y únicamente, deberá agregarle 4 dígitos al inicio de la línea seguidos del texto, por ejemplo: <pre> 0001 program calendario; 0002 uses crt, dos; 0003 0004 const borde = #205#205#205#205#205#205#205#205#205; 0005 const months : array[0..11] of integer = (31, 28, 31, 30, 31, 30, 31, 31, 30, 0006 const year_regular : array[0..11] of integer = (0, 3, 3, 6, 1, 4, 6, 2, 5,</pre> - En los próximos puntos a evaluar de esta rúbrica, cuando se encuentre un incumplimiento , o un error se deberá indicar como error utilizando el siguiente formato: <u>Número del error</u>: la tipificación es a criterio del desarrollador <u>Descripción</u>: corresponde a un error específico del problema encontrado no se debe hacer errores generalizados sino debe considerar ser muy puntual con lo que sucede. Se debe ver de la siguiente manera: Error 200. No encuentra la palabra inicial program - Otra manera de agregar los errores es al final del archivo, pero debe de indicar el número de línea donde encontró el error y respetar lo anterior, ejemplo: Error 200. Línea 0001. No encuentra la palabra inicial program - Considere que en una misma línea puede haber más de un error que debe detectar.
---	--

3

Identificadores: En **PASCAL**, un identificador es un nombre utilizado para identificar variables, funciones, clases u otros elementos en el código.

Los identificadores deben seguir ciertas reglas para ser válidos:

- Inician con la palabra reservada **var** y las mismas deben ubicarse después de declarar correctamente **program** y **uses** y antes del **begin**.
- Comienzan con una letra: Un identificador debe comenzar con una letra (a-z, A-Z).
- Pueden contener letras y guiones bajos: Después del primer carácter, los identificadores pueden contener letras y guiones bajos, **no** puede haber números ni contener otros caracteres especiales.
- La variable es precedida por un espacio luego un símbolo de dos puntos (:), un espacio, luego el tipo de variable y punto y coma, para este caso se valida del igual hacia la izquierda y los únicos tipos de variable que van a validarse es **integer**, **string** y **Word**
- La declaración de la variable finaliza obligatoriamente con punto y coma, no importa si viene espacio entre el tipo de dato y el punto y coma o si viene sin espacio.
- Si la variable usada no existe debe indicar error
- No debe venir después del begin y debe aparecer después de **program**, **uses** y **const**, cualquier aparición errónea es error.
- No puede usar palabras reservadas de **PASCAL** para este proyecto solo validaremos las siguientes palabras reservadas.

ABSOLUTE	DOWNT0	BEGIN	DESTRUCTOR	MOD
AND	ELSE	CASE	EXTERNAL	NOT
ARRAY	END	CONST	DIV	PACKED
ASM	FILE	CONSTRUCTOR	DO	PROCEDURE
FOR	FORWARD	FUNCTION	GOTO	RECORD
IF	IN	OR	PRIVATE	UNTIL
PROGRAM	REPEAT	STRING	THEN	VAR
WHILE	XOR	WITH	TYPE	OF
USES	SET	OBJECT	TO	FOR

- Ejemplos de identificadores válidos:
 - var meses : integer; // Correcto, comienza con la palabra var y la variable comienza con una letra, posee los dos puntos y el tipo integer.
 - var modulo_mes : integer; // Correcto, con la palabra var y la variable comienza con una letra y solo lleva un guion bajo, posee los dos puntos y el tipo integer

	- Ejemplos de identificadores no válidos: - var 123ejemplo :integer; // Incorrecto, no puede comenzar con un número - var +ejemplo:integer; // Incorrecto, el símbolo '+' no está permitido - var repeat : integer; // incorrecto repeat es una palabra reservada - var ejemplo:integer; // incorrecto no respeta el punto que indica que entre los dos puntos y el tipo de datos debe haber un espacio - var días : integer // incorrecto no lleva el punto y coma
4	Inicio del programa Un programa en pascal debe iniciar con la palabra program - Después de program viene un identificador, para efectos de este proyecto este nombre debe calzar con el nombre del archivo .pas a probar sino es un error, esto se refiere a que si el nombre del archivo es calculadora.pas, el nombre que debe llevar a la par de program es calculadora - No debe venir repetido, no puede declararse ni usarse la palabra en ningún otro lado que no sea al inicio - No deben venir líneas, espacios o comentarios antes de program - Finaliza con punto y coma - Entre program y uses no debe venir espacios, comentarios, líneas vacías o tabs - Después de program debe venir la palabra uses, para efectos de este proyecto no se va a validar el comando que este después de uses, se tomara como correcto, pero el estudiante debe validar que venga la palabra uses, un comando y el punto y coma al final. - La ausencia de alguno de estos puntos es un error

5	Constantes • Se declaran con el nombre const • Deben ir después de program, uses y antes de var • No puede aparecer después del begin • Al usarse en el código debe validarse que fue declarada correctamente • Llevan la misma validación de los identificadores en cuanto al nombre de la constante • Se va a considerar que después del identificador debe venir el signo de igual (=) y un valor este puede ser de tipo entero o string, el estudiante puede suponer que el valor va a venir correcto solo debe revisar que cumpla la estructura y finalice en punto y coma • Debe finalizar con punto y coma (;) • Para efectos de este proyecto las constantes ya declaradas en el código que lleven la palabra array como en el caso del código fuente calendario.pas, solo debe validarse que venga la palabra const + identificador valido + comandos + punto y coma. Las demás declaraciones deben seguir las indicaciones anteriores. Ejemplos de constantes correctas ▪ const tipo = 'Mes'; ▪ const mes = 12;
6	Begin y end. • Begin debe aparecer despues de la declaracion de var • No debe tener ninguna otra palabra a la par ni siquiera un comentario en la misma linea • La palabra end., significa el fin del codigo y debe aparecer tal cual viene en el ejemplo end + punto (.) • No debe venir comentarios a la par de end y no puede finalizar con otro carácter que no sea punto • No debe venir líneas extras, ni comentarios despues de end. • La ausencia de alguno de estos puntos es un error

7	Write y WriteLn • Debe aparecer después del comando begin y antes de end. • Tanto el comando write y writeln debe llevar paréntesis de apertura (y paréntesis de cierre) • Después del paréntesis de apertura si va a imprimirse texto debe llevar comillas simples, debe validar que si inicia con comillas termine con comillas • Se debe validar que en los paréntesis exista algún contenido no pueden venir vacíos. • Si imprime un identificador debe validar que el mismo exista • Deben finalizar con punto y coma Ejemplos correctos del uso de ambas ▪ Write ('esto es un ejemplo'); //es correcto porque lleva la estructura como se indico ▪ Write (dia); // es correcto porque la estructura es como se indicó y la variable día existe ▪ Writeln ('Digite el mes'); // es correcto porque viene entre comillas y trae todos sus componentes ▪ Writeln (meses); //es correcto porque viene con sus componentes y meses es una variable que existe Casos incorrectos ▪ Write (); // incorrecto porque no lleva nada entre los paréntesis ▪ Write ; // incorrecto porque no lleva los paréntesis ni contenido ▪ Write (''); // incorrecto porque le falta contenido y comillas ▪ Write (escriba el mes); // incorrecto porque el texto no viene entre comillas ▪ Writeln (); // incorrecto porque en el paréntesis no viene nada ni variable ni texto ▪ Writeln ; // incorrecto porque no vienen los paréntesis por lo consiguiente ni contenido ni comillas ▪ Writeln (; // incorrecto porque le falta el paréntesis de cierre y contenido ▪ Writeln (escriba el mes); // incorrecto porque no viene entre comillas un texto
---	---

8	Comentarios • Se van a manejar dos tipos de comentarios el comentario con doble slash (//) y el comentario entre corchetes ({}). • Cuando se encuentre una línea de comentarios se debe ignorar la línea • Se debe validar que vengan ambos elementos si se usa slash (/), que vengan dos y sin espacios y en el caso de los corchetes que venga el de apertura y cierre • No deben aparecer comentarios después de punto y coma • No deben aparecer comentarios antes de program ni después de end. • No van a realizarse comentarios de doble línea, por ejemplo: <pre>{ hola Es una prueba }</pre>
9	Documentar: Documente en un archivo de texto adicional y con extensión .txt llamado "INFORME.txt", los pasos necesarios para ejecutar su programa, junto con cualquier información adicional que desee clarificar al profesor sobre su proyecto. Asegúrese de verificar y confirmar que estos pasos funcionen correctamente antes de la entrega.
10	Revise las rubricas de evaluación e instrucciones de entrega señaladas

Honestidad Académica



<https://audiovisuales.uned.ac.cr/play/player/23048>

Nota Importante

Cada estudiante es responsable del contenido que entrega, si no es el archivo correcto, no podrá entregarlo posterior a la fecha establecida.

Si el contenido del archivo coincide con algún otro estudiante, o se comprueba que no es de su autoría, se aplicaría lo indicado en la plataforma en el documento [Lineamientos ante casos de plagio](#)

Indicaciones Importantes

- Es obligatorio que incluya todo el directorio donde se encuentra el proyecto Proyecto 1.
- El Proyecto 1 debe estar desarrollado en **(IDE) Apache NetBeans** que es la herramienta oficial del curso.
- El programa debe ser modular, utilizando de la mejor manera funciones definidas por usted.
- Los trabajos deben realizarse en forma individual. Dentro del código del programa debe de indicar la documentación que explique cómo fue realizado el programa.
- Si utiliza código de algún ejemplo del libro, o de otra fuente que no sea de su autoría, debe de indicarlo.
- Comprima todos los archivos en un solo archivo .zip o .rar.
- Nombre del archivo que envía: debe ser nombre y primer apellido del estudiante, y nombre de la tarea. Ejemplo: JuanRojas-tarea1.
- La entrega del Proyecto 1 en las fechas establecidas en la plataforma de aprendizaje en línea Moodle en el apartado que se indique.
- Si no concluyó a tiempo la tarea, debe entregar lo que pudo hacer e incluir una carta explicando las razones por las cuales no finalizó.
- Tenga en consideración que el tutor realizará la evaluación en base al proyecto que suba. Y no se podrá subsanar "errores" o "equivocaciones por versiones o faltas de carpetas" posteriores a la fecha.

Rúbrica de Evaluación

Criterio	Cumple a satisfacción lo indicado en la evaluación	Cumple medianamente en lo indicado en la evaluación	Cumple en contenido y formato, pero los aportes no son significantes	No cumple o no presenta lo solicitado
1. Lectura del archivo: Al leer el archivo su programa debe aceptar espacios en blanco, líneas en blanco, cualquier espacio en general que se encuentre y que se distinga entre mayúsculas y minúsculas. Es decir, el formato original del archivo debe ser respetado en su totalidad. La ejecución del ejecutable .jar debe poder realizarse desde la consola CMD o símbolo del sistema.	5	3	1	0
2. Lectura del archivo: Es importante, que el archivo .pas, no dependa de un nombre en específico para poder ser leído, y que no dependa de una ruta estática o física ni configuraciones personalizadas por el desarrollador	5	3	1	0
3. Lectura del archivo: Donde se encuentre el compilado de su programa .jar en la misma carpeta esté el archivo .pas a leer	5	3	1	0
4. Para el manejo de errores: A partir del archivo que se leyó se debe crear otro archivo con extensión .err	5	3	1	0
5. Para el manejo de errores: El nombre del nuevo archivo debe conservar el nombre original y agregarle un guion con la descripción "-errores.err" el archivo debe verse de esta manera: "nombreArchivo-errores.err" donde "nombreArchivo" depende del nombre que tenga dicho archivo de entrada .pas	5	3	1	0
6. Para el manejo de errores: Se debe respetar el contenido y formato del archivo original .pas, únicamente, deberá agregarle 4 dígitos al inicio de la línea seguidos del texto	5	3	1	0
7. Para el manejo de errores: Se deberá indicar como error utilizando el siguiente formato: - Número del error: la tipificación es a criterio del desarrollador - Descripción: corresponde a un error específico del problema encontrado no se debe hacer errores generalizados sino debe considerar ser muy puntual con lo que sucede	5	3	1	0

8. Identificadores: Comienzan con una letra: Un identificador debe comenzar con una letra (a-z, A-Z) y pueden contener un guion bajo en el medio (_).	15	8	5	0
9. Variables: al declarar una variable debe llevar la palabra var , debe estar después de program , uses , const y antes de begin . No se puede declarar en medio de begin y end y deben terminar con punto y coma. El identificador debe cumplir con lo establecido en el punto 8 y 10	7	5	2	0
10. Palabras Reservadas: No puede usar palabras reservadas de PASCAL	5	3	1	0
11. Inicio del programa: Debe venir la palabra program , debe validar que el nombre del identificador sea igual al del archivo y finalizar con punto y coma	5	3	1	
12. Uses: después de program declarado correctamente debe venir uses un comando y finalizar con punto y coma	5	3	1	0
13. Constantes: debe validar que venga la palabra const , que el identificador sea válido que lleve el signo igual (=) y que este antes de var y después de program y uses .	5	3	1	0
14. Begin: debe venir el comando, el mismo no lleva ninguna otra cosa escrita a la par, no lleva punto y coma para finalizar la línea no debe aparecer antes de los comandos, program , uses , const y var .	5	3	2	0
15. End. Debe aparecer al final del archivo y debe llevar un punto(.), no se acepta ningún otro carácter ni comentarios sobre la misma línea o después de la misma. No puede existir un end. , antes de un begin	5	3	2	0
16. Write y WriteLn: Se debe poner paréntesis de apertura y cierre, al usarse puede venir vacío, con un comando string que debe llevar comillas simples o una variable, la cual debe validarse si fue declarada anteriormente, deben estar entre begin y end solamente.	5	3	1	0

17. Comentarios: No puedes poner comentarios después de punto y coma. Tampoco debes poner comentarios antes de iniciar el programa ni al final de end, si colocas un comentario en un lugar como este, el intérprete generará un error. Se debe validar que los comentarios con doble slash (//) vengan ambos y no lleven espacio, si se usan de corchete debe venir ambos apertura y cierre {}, no van a venir comentarios de doble línea	5	3	1	0
18. Documentar: Documente en un archivo de texto con extensión .txt llamado "INFORME.txt", los pasos necesarios para ejecutar su programa, junto con cualquier información adicional que desee clarificar al profesor sobre su proyecto. Asegúrese de verificar y confirmar que estos pasos funcionen correctamente antes de la entrega.	3	2	1	0
TOTAL	100	60	25	0